

React 기본 개념 학습 가이드

JavaScript 핵심 문법부터 React Hooks, API 연결, 컴포넌트 분리, 라우팅까지 한 번에 훑는 입문 목차형 자료

읽는 법

1. JavaScript 핵심 개념을 먼저 이해합니다.
2. 컴포넌트, JSX, props, state를 React의 중심 축으로 잡습니다.
3. 입력 처리, API 연결, Hooks, 라우팅을 작은 예제로 반복합니다.

목차

React 완전 처음 시작 문서	5
0. 먼저 큰 그림 잡기	5
1. React를 배우기 전에 필요한 JavaScript 최소 문법	5
1.1 값	5
1.2 변수	6
1.3 함수	6
1.4 화살표 함수	7
1.5 객체	7
1.6 배열	8
1.7 조건문	8
1.8 삼항 연산자	9
1.9 map	9
1.10 구조 분해 할당	9
1.11 import와 export	10
2. React란 무엇인가?	10
3. 컴포넌트	11
4. JSX	12
4.1 JSX의 기본 규칙	12
4.2 JSX에서 JavaScript 표현식 쓰기	13
5. props	14
6. 이벤트	15
7. state	16
7.1 문자열 state	17
7.2 boolean state	17
8. 조건부 렌더링	17
8.1 if 사용	18
8.2 삼항 연산자 사용	18
8.3 && 사용	18
9. 목록 렌더링	18
10. state가 배열일 때	19
11. state가 객체일 때	20
12. 입력 폼	21
13. useEffect	22
14. 컴포넌트 나누기	23
15. 파일 나누기	24
16. 스타일링	24
17. 작은 완성 예제: 카운터	25

18. 작은 완성 예제: 할 일 목록	26
19. React 앱 실행 흐름	27
20. 개발 환경 만들기	28
21. 자주 만나는 에러	29
21.1 컴포넌트 이름을 소문자로 시작함	29
21.2 JSX에서 여러 태그를 바로 return함	29
21.3 class를 사용함	29
21.4 이벤트 함수를 바로 실행함	29
21.5 map에서 key를 빼먹음	30
22. React 문법 핵심 요약	30
컴포넌트	30
JSX에서 변수 보여주기	30
props	30
이벤트	30
state	31
조건부 렌더링	31
목록 렌더링	31
23. 추천 학습 순서	31
24. 연습 문제	31
연습 1: 자기소개 컴포넌트	31
연습 2: props 사용하기	31
연습 3: 카운터 만들기	32
연습 4: 과일 목록 만들기	32
연습 5: 입력창 만들기	32
25. 마지막으로 꼭 기억할 것	32
26. 체크리스트로 보는 학습 범위	33
27. JavaScript 기본 문법 보강	33
27.1 switch	33
27.2 for	34
27.3 for...of	34
27.4 return	34
27.5 템플릿 리터럴	34
28. 배열과 객체 다루기 보강	35
28.1 forEach	35
28.2 find	35
28.3 reduce	35
28.4 객체 접근	36
28.5 rest 문법	36
29. 비동기 처리	37

29.1 동기과 비동기	37
29.2 Promise	37
29.3 async와 await	38
29.4 try/catch	38
29.5 JSON	38
30. HTTP 요청과 응답	39
30.1 GET 요청	39
30.2 POST 요청	39
30.3 response.ok	40
31. React에서 API 연결하기	40
31.1 React에서 FastAPI 호출 예시	41
32. axios로 API 호출하기	42
33. DOM 기초와 React 이벤트	43
34. 입력 처리 확장	44
34.1 textarea	44
34.2 select	44
34.3 여러 input 처리	45
35. React Hooks 보강	46
35.1 useRef	46
35.2 custom hook	47
36. 컴포넌트 분리 보강	48
36.1 children	48
36.2 상위 컴포넌트와 하위 컴포넌트	48
36.3 상태 끌어올리기	48
37. 라우팅	50
37.1 설치	50
37.2 BrowserRouter, Routes, Route	50
37.3 Link	51
37.4 useNavigate	51
37.5 useParams	51
37.6 라우팅 파일 분리 예시	52
38. 전체 학습 순서 업데이트	52
참고 자료	53

React 완전 처음 시작 문서

작성일: 2026-06-06 최종 보강일: 2026-06-07 기준: React 공식 문서 v19.2의 기본 학습 흐름, React Router v7 기본 라우팅 흐름

이 문서는 JavaScript도, React도 처음 보는 사람을 위한 입문서입니다. 목표는 React 문법을 외우는 것이 아니라, "화면을 어떻게 코드로 나누고 바꾸는지"를 이해하는 것입니다.

0. 먼저 큰 그림 잡기

웹 화면은 보통 세 가지로 만들어집니다.

```
HTML    화면의 뼈대
CSS     화면의 모양
JavaScript 화면의 동작
```

예를 들어 버튼 하나를 생각해 봅시다.

```
<button>좋아요</button>
```

이건 HTML입니다. 버튼이 화면에 보이게 합니다.

```
button {
  background: blue;
  color: white;
}
```

이건 CSS입니다. 버튼의 색과 모양을 바꿉니다.

```
button.addEventListener("click", function () {
  alert("눌렀어요!");
});
```

이건 JavaScript입니다. 버튼을 눌렀을 때 어떤 일이 일어나게 합니다.

React는 여기서 JavaScript 쪽에 가까운 도구입니다. 정확히는 "화면을 여러 조각으로 나누고, 데이터가 바뀌면 화면도 자동으로 다시 그리게 도와주는 라이브러리"입니다.

1. React를 배우기 전에 필요한 JavaScript 최소 문법

React는 JavaScript 위에서 동작합니다. 그래서 React를 이해하려면 JavaScript의 모든 문법을 알 필요는 없지만, 아래 문법은 꼭 알아야 합니다.

1.1 값

프로그래밍에서 값은 코드가 다루는 재료입니다.

```
10; // 숫자
"안녕하세요"; // 문자열
true; // 참
false; // 거짓
```

```
null; // 비어 있음
undefined; // 아직 값이 없음
```

문자열은 글자입니다. 보통 따옴표로 감쌉니다.

```
"React";
'JavaScript';
```

React 예제에서는 큰따옴표와 작은따옴표가 섞여 나올 수 있습니다. 둘 다 문자열을 만들 수 있습니다.

1.2 변수

변수는 값에 이름을 붙이는 것입니다.

```
const name = "지민";
const age = 20;

console.log(name);
console.log(age);
```

`const`는 "이 이름에 값을 담겠다"는 뜻입니다. React에서는 대부분 `const`를 많이 씁니다.

```
const message = "안녕하세요";
```

이 코드는 이렇게 읽으면 됩니다.

```
message라는 이름에 "안녕하세요"라는 값을 담는다.
```

`let`은 나중에 값이 바뀔 수 있는 변수를 만들 때 씁니다.

```
let count = 0;
count = count + 1;
```

처음에는 이렇게 기억하면 충분합니다.

```
const 다시 대입하지 않을 값
let 나중에 바뀔 수 있는 값
```

1.3 함수

함수는 여러 줄의 코드를 이름 붙여 묶은 것입니다.

```
function sayHello() {
  console.log("안녕하세요");
}

sayHello();
```

이 코드는 이렇게 읽습니다.

```
sayHello라는 함수를 만든다.
sayHello를 실행한다.
```

함수는 값을 받을 수도 있습니다. 이때 받는 값을 매개변수라고 합니다.

```
function greet(name) {  
  console.log(name + "님, 안녕하세요!");  
}  
  
greet("수빈");  
greet("민재");
```

함수는 값을 돌려줄 수도 있습니다. 이때 `return`을 씁니다.

```
function add(a, b) {  
  return a + b;  
}  
  
const result = add(2, 3);  
console.log(result); // 5
```

React 컴포넌트도 결국 함수입니다. 이 점이 가장 중요합니다.

1.4 화살표 함수

React 예제에는 이런 함수 문법도 자주 나옵니다.

```
const sayHello = () => {  
  console.log("안녕하세요");  
};
```

이건 아래와 비슷한 뜻입니다.

```
function sayHello() {  
  console.log("안녕하세요");  
}
```

값을 받는 화살표 함수는 이렇게 씁니다.

```
const greet = (name) => {  
  console.log(name + "님, 안녕하세요!");  
};
```

한 줄로 바로 값을 돌려줄 수도 있습니다.

```
const add = (a, b) => a + b;
```

처음에는 `function` 문법을 먼저 이해하고, 화살표 함수는 "함수를 짧게 쓰는 방식" 정도로 보면 됩니다.

1.5 객체

객체는 여러 값을 이름표와 함께 묶은 것입니다.

```
const user = {  
  name: "지민",  
  age: 20,
```

```
isStudent: true,  
};  
  
console.log(user.name); // 지민  
console.log(user.age); // 20
```

객체는 React에서 아주 자주 씁니다. 사용자 정보, 상품 정보, 글 정보 등을 표현하기 좋습니다.

```
const product = {  
  id: 1,  
  title: "노트북",  
  price: 1200000,  
};
```

값을 꺼낼 때는 점을 씁니다.

```
product.title;  
product.price;
```

1.6 배열

배열은 여러 값을 순서대로 담는 목록입니다.

```
const fruits = ["사과", "바나나", "딸기"];  
  
console.log(fruits[0]); // 사과  
console.log(fruits[1]); // 바나나
```

배열은 0번부터 시작합니다.

객체 배열도 많이 씁니다.

```
const todos = [  
  { id: 1, text: "React 공부하기", done: false },  
  { id: 2, text: "점심 먹기", done: true },  
  { id: 3, text: "산책하기", done: false },  
];
```

React에서 목록을 화면에 그릴 때 배열이 거의 항상 등장합니다.

1.7 조건문

조건문은 상황에 따라 다른 코드를 실행합니다.

```
const isLoggedIn = true;  
  
if (isLoggedIn) {  
  console.log("환영합니다");  
} else {  
  console.log("로그인이 필요합니다");  
}
```

React에서는 조건에 따라 다른 화면을 보여줄 때 씁니다.


```
if (isLoggedIn) {  
  return <p>환영합니다</p>;  
}  
  
return <p>로그인이 필요합니다</p>;
```

1.8 삼항 연산자

삼항 연산자는 짧은 조건문입니다.

```
const age = 20;  
const message = age >= 19 ? "성인입니다" : "미성년자입니다";
```

구조는 이렇습니다.

```
조건 ? 조건이 참일 때 값 : 조건이 거짓일 때 값
```

React JSX 안에서는 if를 바로 쓰기 어렵기 때문에 삼항 연산자를 자주 씁니다.

```
<p>{isLoggedIn ? "로그인됨" : "로그인 필요"}</p>
```

1.9 map

map은 배열의 각 항목을 다른 모양의 태그로 바꿀 때 씁니다.

```
const fruits = ["사과", "바나나", "딸기"];  
  
const list = fruits.map((fruit) => {  
  return <li>{fruit}</li>;  
});
```

짧게 쓰면 이렇게 됩니다.

```
const list = fruits.map((fruit) => <li>{fruit}</li>);
```

처음에는 짧은 문법보다 긴 문법으로 읽는 연습을 추천합니다.

1.10 구조 분해 할당

구조 분해 할당은 객체나 배열에서 값을 쉽게 꺼내는 문법입니다.

객체 예시:

```
const user = {  
  name: "지민",  
  age: 20,  
};  
  
const { name, age } = user;  
  
console.log(name); // 지민  
console.log(age); // 20
```

배열 예시:

```
const colors = ["red", "blue"];

const [first, second] = colors;

console.log(first); // red
console.log(second); // blue
```

React의 `useState`에서 배열 구조 분해 할당이 바로 등장합니다.

```
const [count, setCount] = useState(0);
```

이 문장은 처음에는 낯설지만, 이렇게 읽으면 됩니다.

count라는 현재 값과
setCount라는 값을 바꾸는 함수를
useState(0)에서 꺼낸다.

1.11 import와 export

React 프로젝트는 파일을 나눠서 만듭니다. 한 파일에서 만든 것을 다른 파일에서 쓰려면 `export`와 `import`를 씁니다.

```
// Button.jsx
export default function Button() {
  return <button>클릭</button>;
}
```

```
// App.jsx
import Button from "./Button";

export default function App() {
  return <Button />;
}
```

이렇게 읽으면 됩니다.

Button.jsx에서 Button 컴포넌트를 내보낸다.
App.jsx에서 Button 컴포넌트를 가져와서 쓴다.

2. React란 무엇인가?

React는 UI를 만들기 위한 JavaScript 라이브러리입니다.

UI는 User Interface의 줄임말입니다. 사용자가 보는 화면, 버튼, 입력창, 목록, 카드 같은 것들을 말합니다.

React의 핵심 생각은 간단합니다.

화면을 작은 컴포넌트로 나눈다.
각 컴포넌트는 JavaScript 함수처럼 만든다.
데이터가 바뀌면 React가 화면을 다시 그린다.

예를 들어 쇼핑몰 화면을 생각해 봅시다.

```
App
  Header
  ProductList
    ProductCard
    ProductCard
    ProductCard
  Footer
```

React에서는 이런 화면 조각을 컴포넌트라고 부릅니다.

3. 컴포넌트

React 컴포넌트는 화면 조각을 돌려주는 JavaScript 함수입니다.

```
function Welcome() {
  return <h1>안녕하세요</h1>;
}
```

이 함수는 HTML처럼 생긴 `<h1>안녕하세요</h1>`를 돌려줍니다.

컴포넌트 이름은 반드시 대문자로 시작해야 합니다.

```
function Welcome() {
  return <h1>안녕하세요</h1>;
}
```

좋은 예:

```
function ProfileCard() {
  return <div>프로필 카드</div>;
}
```

나쁜 예:

```
function profileCard() {
  return <div>프로필 카드</div>;
}
```

HTML 태그는 소문자로 씁니다.

```
<div></div>
<button></button>
<p></p>
```

내가 만든 React 컴포넌트는 대문자로 씁니다.

```
<ProfileCard />
<Welcome />
```

4. JSX

JSX는 JavaScript 안에서 HTML처럼 화면을 쓰는 문법입니다.

```
function App() {  
  return (  
    <div>  
      <h1>나의 첫 React</h1>  
      <p>React를 배우는 중입니다.</p>  
    </div>  
  );  
}
```

겉으로는 HTML처럼 보이지만 실제로는 JavaScript 문법입니다.

4.1 JSX의 기본 규칙

첫 번째, 하나의 컴포넌트는 하나의 부모 태그만 return해야 합니다.

틀린 예:

```
function App() {  
  return (  
    <h1>제목</h1>  
    <p>본문</p>  
  );  
}
```

맞는 예:

```
function App() {  
  return (  
    <div>  
      <h1>제목</h1>  
      <p>본문</p>  
    </div>  
  );  
}
```

div를 추가하고 싶지 않을 때는 빈 태그를 쓸 수 있습니다. 이것을 Fragment라고 부릅니다.

```
function App() {  
  return (  
    <>  
      <h1>제목</h1>  
      <p>본문</p>  
    </>  
  );  
}
```

두 번째, 태그는 반드시 닫아야 합니다.

```
  
<br />  
<input />
```

세 번째, HTML의 `class` 대신 `className`을 씁니다.

```
<div className="card">카드</div>
```

네 번째, JavaScript 값을 JSX 안에 넣을 때는 `{}`를 씁니다.

```
function App() {  
  const name = "지민";  
  
  return <h1>{name}님 안녕하세요</h1>;  
}
```

4.2 JSX에서 JavaScript 표현식 쓰기

중괄호 안에는 JavaScript 표현식을 넣을 수 있습니다.

```
function App() {  
  const price = 10000;  
  
  return <p>가격: {price}원</p>;  
}
```

계산도 가능합니다.

```
<p>총액: {price * 2}원</p>
```

함수 호출도 가능합니다.

```
function formatPrice(price) {  
  return price.toLocaleString() + "원";  
}  
  
function App() {  
  return <p>{formatPrice(1200000)}</p>;  
}
```

객체 값을 꺼낼 수도 있습니다.

```
function App() {  
  const user = {  
    name: "지민",  
    age: 20,  
  };  
  
  return (  
    <div>  
      <p>이름: {user.name}</p>  
      <p>나이: {user.age}</p>  
    </div>  
  );  
}
```

주의할 점: JSX 중괄호 안에는 `if` 문 자체를 바로 넣을 수 없습니다.

틀린 예:

```
<p>{if (isLoggedIn) { "로그인됨" }}</p>
```

대신 삼항 연산자를 씁니다.

```
<p>{isLoggedIn ? "로그인됨" : "로그인 필요"}</p>
```

5. props

props는 부모 컴포넌트가 자식 컴포넌트에게 전달하는 값입니다.

먼저 같은 내용을 반복하는 코드를 봅시다.

```
function App() {
  return (
    <div>
      <div>
        <h2>지민</h2>
        <p>프론트엔드 공부 중</p>
      </div>

      <div>
        <h2>수빈</h2>
        <p>백엔드 공부 중</p>
      </div>
    </div>
  );
}
```

반복되는 프로필 카드를 컴포넌트로 만들 수 있습니다.

```
function ProfileCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>{props.description}</p>
    </div>
  );
}

function App() {
  return (
    <div>
      <ProfileCard name="지민" description="프론트엔드 공부 중" />
      <ProfileCard name="수빈" description="백엔드 공부 중" />
    </div>
  );
}
```

이 코드는 이렇게 읽습니다.

App이 ProfileCard에게 name과 description을 전달한다.
ProfileCard는 props.name, props.description으로 그 값을 사용한다.

보통은 구조 분해 할당으로 더 짧게 씁니다.

```
function ProfileCard({ name, description }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{description}</p>
    </div>
  );
}
```

props는 읽기 전용입니다. 자식 컴포넌트가 props를 직접 바꾸면 안 됩니다.

6. 이벤트

이벤트는 사용자가 화면에서 하는 행동입니다.

클릭
입력
마우스 올리기
폼 제출

React에서는 이벤트 이름을 camelCase로 씁니다.

onClick
onChange
onSubmit

버튼 클릭 예시:

```
function App() {
  function handleClick() {
    alert("버튼을 눌렀어요!");
  }

  return <button onClick={handleClick}>클릭</button>;
}
```

중요한 차이를 봅시다.

```
<button onClick={handleClick}>클릭</button>
```

이건 맞습니다. 클릭했을 때 handleClick을 실행하라는 뜻입니다.

```
<button onClick={handleClick()}>클릭</button>
```

이건 대부분 틀립니다. 화면이 그려질 때 바로 handleClick()을 실행해 버립니다.

값을 전달해야 할 때는 화살표 함수로 감쌉니다.

```
function App() {
  function sayHello(name) {
    alert(name + "님 안녕하세요!");
  }
}
```

```
return <button onClick={() => sayHello("지민")}>인사하기</button>;
}
```

7. state

state는 컴포넌트가 기억해야 하는 값입니다.

예를 들어 카운터를 생각해 봅시다.

현재 숫자: 0
버튼을 누르면 1 증가
다시 누르면 2
다시 누르면 3

이 숫자는 화면이 기억해야 합니다. 이런 값을 state라고 합니다.

React에서 state를 쓰려면 `useState`를 import합니다.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }

  return (
    <div>
      <p>현재 숫자: {count}</p>
      <button onClick={handleClick}>+1</button>
    </div>
  );
}
```

이 한 줄이 중요합니다.

```
const [count, setCount] = useState(0);
```

이렇게 읽으면 됩니다.

count: 현재 값
setCount: count를 바꾸는 함수
useState(0): 처음 값은 0

state를 바꿀 때는 직접 대입하지 않습니다.

틀린 예:

```
count = count + 1;
```

맞는 예:


```
setCount(count + 1);
```

왜 직접 바꾸면 안 될까요? React는 `setCount` 같은 state 변경 함수를 통해 값이 바뀐 것을 알고 화면을 다시 그립니다.

7.1 문자열 state

입력창 값을 기억할 수도 있습니다.

```
import { useState } from "react";

function NameInput() {
  const [name, setName] = useState("");

  function handleChange(event) {
    setName(event.target.value);
  }

  return (
    <div>
      <input value={name} onChange={handleChange} />
      <p>입력한 이름: {name}</p>
    </div>
  );
}
```

`event.target.value`는 "사용자가 입력한 현재 값"이라고 이해하면 됩니다.

7.2 boolean state

참/거짓 값도 state로 많이 씁니다.

```
import { useState } from "react";

function ToggleButton() {
  const [isOn, setIsOn] = useState(false);

  function handleClick() {
    setIsOn(!isOn);
  }

  return (
    <button onClick={handleClick}>
      {isOn ? "켜짐" : "꺼짐"}
    </button>
  );
}
```

`!isOn`은 참이면 거짓으로, 거짓이면 참으로 바꿉니다.

8. 조건부 렌더링

렌더링은 React가 화면을 그리는 것을 말합니다.

조건부 렌더링은 조건에 따라 다른 화면을 보여주는 것입니다.

8.1 if 사용

```
function Greeting({ isLoggedIn }) {  
  if (isLoggedIn) {  
    return <p>환영합니다!</p>;  
  }  
  
  return <p>로그인이 필요합니다.</p>;  
}
```

8.2 삼항 연산자 사용

```
function Greeting({ isLoggedIn }) {  
  return (  
    <p>  
      {isLoggedIn ? "환영합니다!" : "로그인이 필요합니다."}  
    </p>  
  );  
}
```

8.3 && 사용

특정 조건이 참일 때만 보여주고 싶다면 &&를 씁니다.

```
function Notice({ hasNewMessage }) {  
  return (  
    <div>  
      <h1>마이페이지</h1>  
      {hasNewMessage && <p>새 메시지가 있습니다.</p>}  
    </div>  
  );  
}
```

이 코드는 이렇게 읽습니다.

hasNewMessage가 true이면 <p>새 메시지가 있습니다.</p>를 보여준다.
false이면 아무것도 보여주지 않는다.

9. 목록 렌더링

배열 데이터를 화면 목록으로 보여줄 때 map을 씁니다.

```
function FruitList() {  
  const fruits = ["사과", "바나나", "딸기"];  
  
  return (  
    <ul>  
      {fruits.map((fruit) => (  
        <li>{fruit}</li>  
      ))}  
    </ul>  
  );  
}
```

하지만 React에서는 목록을 그릴 때 각 항목에 `key`를 넣어야 합니다.

```
function FruitList() {
  const fruits = [
    { id: 1, name: "사과" },
    { id: 2, name: "바나나" },
    { id: 3, name: "딸기" },
  ];

  return (
    <ul>
      {fruits.map((fruit) => (
        <li key={fruit.id}>{fruit.name}</li>
      ))}
    </ul>
  );
}
```

`key`는 React가 각 항목을 구분하기 위한 이름표입니다. 보통 데이터의 `id`를 씁니다.

10. state가 배열일 때

할 일 목록처럼 여러 항목을 추가하고 삭제하려면 배열 `state`를 씁니다.

```
import { useState } from "react";

function TodoList() {
  const [todos, setTodos] = useState([
    { id: 1, text: "React 공부하기" },
    { id: 2, text: "운동하기" },
  ]);

  function addTodo() {
    const newTodo = {
      id: Date.now(),
      text: "새 할 일",
    };

    setTodos([...todos, newTodo]);
  }

  return (
    <div>
      <button onClick={addTodo}>추가</button>

      <ul>
        {todos.map((todo) => (
          <li key={todo.id}>{todo.text}</li>
        ))}
      </ul>
    </div>
  );
}
```

이 문법이 중요합니다.

```
setTodos([...todos, newTodo]);
```

...todos는 기존 배열의 항목들을 펼쳐 넣는 문법입니다. 기존 배열을 직접 수정하지 않고, 새 배열을 만들어 state를 바꿉니다.

React에서는 state 배열을 직접 바꾸는 방식을 피해야 합니다.

피해야 할 예:

```
todos.push(newTodo);
setTodos(todos);
```

권장 예:

```
setTodos([...todos, newTodo]);
```

삭제는 filter를 자주 씁니다.

```
function deleteTodo(id) {
  const nextTodos = todos.filter((todo) => todo.id !== id);
  setTodos(nextTodos);
}
```

filter는 조건에 맞는 항목만 남긴 새 배열을 만듭니다.

11. state가 객체일 때

사용자 프로필 같은 데이터는 객체 state로 표현할 수 있습니다.

```
import { useState } from "react";

function ProfileEditor() {
  const [user, setUser] = useState({
    name: "지민",
    age: 20,
  });

  function changeName() {
    setUser({
      ...user,
      name: "수빈",
    });
  }

  return (
    <div>
      <p>이름: {user.name}</p>
      <p>나이: {user.age}</p>
      <button onClick={changeName}>이름 변경</button>
    </div>
  );
}
```

이 문법이 중요합니다.

```
setUser({
  ...user,
```

```
name: "수빈",  
});
```

이렇게 읽으면 됩니다.

기존 user 내용을 그대로 복사하고
name만 "수빈"으로 바꾼 새 객체를 만든다.

state 객체도 직접 바꾸면 안 됩니다.

피해야 할 예:

```
user.name = "수빈";  
setUser(user);
```

권장 예:

```
setUser({  
  ...user,  
  name: "수빈",  
});
```

12. 입력 폼

React에서 입력창을 다룰 때는 보통 state와 연결합니다.

```
import { useState } from "react";  
  
function SignupForm() {  
  const [email, setEmail] = useState("");  
  
  function handleChange(event) {  
    setEmail(event.target.value);  
  }  
  
  function handleSubmit(event) {  
    event.preventDefault();  
    alert("가입 이메일: " + email);  
  }  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <input  
        value={email}  
        onChange={handleChange}  
        placeholder="이메일"  
      />  
      <button type="submit">가입</button>  
    </form>  
  );  
}
```

중요한 부분은 세 가지입니다.

```
value={email}
```

입력창에 보여줄 값을 state와 연결합니다.

```
onChange={handleChange}
```

사용자가 입력할 때마다 state를 바꿉니다.

```
event.preventDefault();
```

폼 제출 시 브라우저가 페이지를 새로고침하는 기본 동작을 막습니다.

13. useEffect

`useEffect`는 컴포넌트가 화면에 나타난 뒤에 어떤 일을 실행하고 싶을 때 사용합니다.

예를 들어 화면 제목을 바꾸고 싶을 때:

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    document.title = "React 공부 중";
  }, []);

  return <h1>안녕하세요</h1>;
}
```

구조는 이렇습니다.

```
useEffect(() => {
  실행할 코드
}, [의존성]);
```

두 번째 자리에 있는 배열을 의존성 배열이라고 합니다.

```
useEffect(() => {
  console.log("처음 한 번만 실행");
}, []);
```

빈 배열이면 컴포넌트가 처음 화면에 나타난 뒤 한 번 실행됩니다.

```
useEffect(() => {
  console.log("count가 바뀔 때마다 실행");
}, [count]);
```

`[count]`이면 `count`가 바뀔 때마다 실행됩니다.

처음에는 `useEffect`를 너무 많이 쓰려고 하지 않아도 됩니다. 버튼 클릭, 입력값 변경, 단순 계산은 대부분 `useEffect` 없이 처리할 수 있습니다.

14. 컴포넌트 나누기

처음에는 모든 코드를 `App.jsx`에 써도 됩니다. 하지만 코드가 길어지면 컴포넌트로 나눕니다.

나누기 전:

```
function App() {
  return (
    <div>
      <header>
        <h1>쇼핑몰</h1>
      </header>

      <main>
        <section>
          <h2>노트북</h2>
          <p>가격: 1200000원</p>
        </section>
      </main>
    </div>
  );
}
```

나눈 후:

```
function Header() {
  return (
    <header>
      <h1>쇼핑몰</h1>
    </header>
  );
}

function ProductCard() {
  return (
    <section>
      <h2>노트북</h2>
      <p>가격: 1200000원</p>
    </section>
  );
}

function App() {
  return (
    <div>
      <Header />
      <main>
        <ProductCard />
      </main>
    </div>
  );
}
```

좋은 컴포넌트 분리 기준:

- 반복되는 UI인가?
- 이름 붙일 수 있는 화면 조각인가?
- 코드가 너무 길어졌는가?
- 다른 곳에서도 쓸 수 있는가?

15. 파일 나누기

작은 프로젝트에서는 이런 구조를 많이 씁니다.

```
src/  
  App.jsx  
  main.jsx  
  components/  
    Header.jsx  
    ProductCard.jsx
```

ProductCard.jsx:

```
export default function ProductCard({ title, price }) {  
  return (  
    <section>  
      <h2>{title}</h2>  
      <p>가격: {price}원</p>  
    </section>  
  );  
}
```

App.jsx:

```
import ProductCard from "../components/ProductCard";  
  
export default function App() {  
  return (  
    <main>  
      <ProductCard title="노트북" price={1200000} />  
      <ProductCard title="키보드" price={80000} />  
    </main>  
  );  
}
```

16. 스타일링

React에서 CSS class를 줄 때는 `className`을 씁니다.

```
function Button() {  
  return <button className="primary-button">저장</button>;  
}
```

CSS 파일:

```
.primary-button {  
  background: #2563eb;  
  color: white;  
  border: none;  
  padding: 8px 12px;  
  border-radius: 6px;  
}
```


조건에 따라 className을 바꿀 수도 있습니다.

```
function Message({ isError }) {
  return (
    <p className={isError ? "message error" : "message"}>
      안내 메시지
    </p>
  );
}
```

간단한 인라인 스타일도 가능합니다.

```
function Box() {
  return (
    <div
      style={{
        backgroundColor: "black",
        color: "white",
        padding: "16px",
      }}
    >
      박스
    </div>
  );
}
```

주의할 점: JSX의 인라인 스타일은 문자열이 아니라 객체입니다.

```
style={{
  backgroundColor: "black",
}}
```

CSS의 `background-color`는 JavaScript 객체에서는 `backgroundColor`처럼 씁니다.

17. 작은 완성 예제: 카운터

아래 예제는 React의 핵심 문법을 많이 포함합니다.

```
import { useState } from "react";

export default function App() {
  const [count, setCount] = useState(0);

  function increase() {
    setCount(count + 1);
  }

  function decrease() {
    setCount(count - 1);
  }

  function reset() {
    setCount(0);
  }

  return (
```

```

<main>
  <h1>카운터</h1>
  <p>현재 숫자: {count}</p>

  <button onClick={increase}>+ 1</button>
  <button onClick={decrease}>- 1</button>
  <button onClick={reset}>초기화</button>
</main>
);
}

```

읽는 순서:

1. useState를 가져온다.
2. count state를 만든다. 처음 값은 0이다.
3. 버튼을 누르면 setCount로 count를 바꾼다.
4. count가 바뀌면 화면의 {count} 부분도 바뀐다.

18. 작은 완성 예제: 할 일 목록

이 예제는 입력, state, 배열, map, 이벤트를 함께 사용합니다.

```

import { useState } from "react";

export default function App() {
  const [text, setText] = useState("");
  const [todos, setTodos] = useState([]);

  function handleChange(event) {
    setText(event.target.value);
  }

  function addTodo(event) {
    event.preventDefault();

    if (text.trim() === "") {
      return;
    }

    const newTodo = {
      id: Date.now(),
      text: text,
    };

    setTodos([...todos, newTodo]);
    setText("");
  }

  function deleteTodo(id) {
    const nextTodos = todos.filter((todo) => todo.id !== id);
    setTodos(nextTodos);
  }

  return (
    <main>
      <h1>할 일 목록</h1>

```

```

<form onSubmit={addTodo}>
  <input
    value={text}
    onChange={handleChange}
    placeholder="할 일을 입력하세요"
  />
  <button type="submit">추가</button>
</form>

<ul>
  {todos.map((todo) => (
    <li key={todo.id}>
      {todo.text}
      <button onClick={() => deleteTodo(todo.id)}>삭제</button>
    </li>
  ))}
</ul>
</main>
);
}

```

이 예제에서 꼭 이해해야 할 부분:

```
const [text, setText] = useState("");
```

입력창에 적는 글자를 기억합니다.

```
const [todos, setTodos] = useState([]);
```

할 일 목록을 배열로 기억합니다.

```
setTodos([...todos, newTodo]);
```

기존 목록에 새 할 일을 추가한 새 배열을 만듭니다.

```

todos.map((todo) => (
  <li key={todo.id}>{todo.text}</li>
))

```

할 일 배열을 `<li>` 목록으로 바꿉니다.

```
onClick={() => deleteTodo(todo.id)}
```

버튼을 클릭했을 때 해당 할 일의 id를 넘겨 삭제합니다.

19. React 앱 실행 흐름

보통 React 프로젝트는 `main.jsx`에서 시작합니다.

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

```

```
createRoot(document.getElementById("root")).render(  
  <StrictMode>  
    <App />  
  </StrictMode>  
);
```

처음 보면 어렵지만 이렇게 이해하면 됩니다.

HTML에 있는 id가 root인 자리 찾기
그 자리에 App 컴포넌트 그리기

App.jsx는 실제 화면의 시작 컴포넌트입니다.

```
export default function App() {  
  return <h1>나의 앱</h1>;  
}
```

20. 개발 환경 만들기

React를 로컬에서 연습할 때는 Vite를 많이 씁니다.

Node.js가 설치되어 있다면 터미널에서 아래 명령을 실행합니다.

```
npm create vite@latest my-react-app
```

선택 화면이 나오면 보통 이렇게 고릅니다.

Framework: React
Variant: JavaScript

그 다음:

```
cd my-react-app  
npm install  
npm run dev
```

브라우저에서 표시되는 주소로 들어가면 React 앱을 볼 수 있습니다.

처음 볼 파일:

```
src/App.jsx  
src/main.jsx  
src/App.css
```

처음에는 App.jsx만 많이 수정해도 충분합니다.

21. 자주 만나는 에러

21.1 컴포넌트 이름을 소문자로 시작함

문제:

```
function profile() {  
  return <div>프로필</div>;  
}
```

해결:

```
function Profile() {  
  return <div>프로필</div>;  
}
```

컴포넌트 이름은 대문자로 시작합니다.

21.2 JSX에서 여러 태그를 바로 return함

문제:

```
return (  
  <h1>제목</h1>  
  <p>본문</p>  
);
```

해결:

```
return (  
  <>  
    <h1>제목</h1>  
    <p>본문</p>  
  </>  
);
```

21.3 class를 사용함

문제:

```
<div class="card">카드</div>
```

해결:

```
<div className="card">카드</div>
```

21.4 이벤트 함수를 바로 실행함

문제:

```
<button onClick={handleClick()}>클릭</button>
```

해결:

```
<button onClick={handleClick}>클릭 </button>
```

값을 넘겨야 한다면:

```
<button onClick={() => handleClick(id)}>클릭 </button>
```

21.5 map에서 key를 빼먹음

문제:

```
{todos.map((todo) => (  
  <li>{todo.text}</li>  
))}
```

해결:

```
{todos.map((todo) => (  
  <li key={todo.id}>{todo.text}</li>  
))}
```

22. React 문법 핵심 요약

컴포넌트

```
function App() {  
  return <h1>안녕하세요</h1>;  
}
```

JSX에서 변수 보여주기

```
const name = "지민";  
  
return <p>{name}</p>;
```

props

```
function Hello({ name }) {  
  return <p>{name}님 안녕하세요</p>;  
}  
  
<Hello name="지민" />
```

이벤트

```
function handleClick() {  
  alert("클릭");  
}  
  
<button onClick={handleClick}>클릭 </button>
```

state

```
const [count, setCount] = useState(0);

setCount(count + 1);
```

조건부 렌더링

```
{isLoggedIn ? <p>환영합니다</p> : <p>로그인 필요</p>}
```

목록 렌더링

```
{items.map((item) => (
  <li key={item.id}>{item.name}</li>
))}
```

23. 추천 학습 순서

처음부터 모든 것을 완벽히 이해하려고 하면 어렵습니다. 아래 순서대로 작게 반복하는 것이 좋습니다.

1. JavaScript 변수, 함수, 객체, 배열 이해
2. JSX 규칙 익히기
3. 컴포넌트 만들기
4. props로 값 전달하기
5. 버튼 클릭 이벤트 만들기
6. useState로 화면 바꾸기
7. map으로 목록 그리기
8. 입력창과 form 다루기
9. 작은 todo 앱 만들기
10. 파일을 컴포넌트별로 나누기

24. 연습 문제

연습 1: 자기소개 컴포넌트

`Profile` 컴포넌트를 만들고 이름, 나이, 좋아하는 것을 화면에 표시해 보세요.

힌트:

```
function Profile() {
  const name = "지민";

  return <h1>{name}</h1>;
}
```

연습 2: props 사용하기

`ProfileCard` 컴포넌트가 `name`, `job`을 props로 받아서 보여주게 만들어 보세요.

예상 사용법:

```
<ProfileCard name="지민" job="프론트엔드 개발자" />
<ProfileCard name="수빈" job="디자이너" />
```

연습 3: 카운터 만들기

버튼을 누르면 숫자가 1씩 증가하는 카운터를 만들어 보세요.

필요한 문법:

```
const [count, setCount] = useState(0);
```

연습 4: 과일 목록 만들기

아래 배열을 `<ul>` 목록으로 보여주세요.

```
const fruits = [
  { id: 1, name: "사과" },
  { id: 2, name: "바나나" },
  { id: 3, name: "딸기" },
];
```

연습 5: 입력창 만들기

입력창에 글자를 입력하면 아래에 그대로 표시되게 만들어 보세요.

필요한 문법:

```
const [text, setText] = useState("");
```

25. 마지막으로 꼭 기억할 것

React의 기본은 거의 아래 문장으로 정리됩니다.

컴포넌트는 화면을 돌려주는 함수다.
props는 부모가 자식에게 주는 값이다.
state는 컴포넌트가 기억하고 바꾸는 값이다.
이벤트는 사용자의 행동에 반응하는 코드다.
배열은 map으로 화면 목록으로 바꾼다.
값이 바뀌면 React가 화면을 다시 그린다.

처음에는 문법이 낯설어서 어렵게 느껴집니다. 하지만 React 코드를 읽을 때 아래 순서로 보면 훨씬 덜 복잡합니다.

1. 어떤 컴포넌트인가?
2. props로 무엇을 받는가?
3. state로 무엇을 기억하는가?
4. 어떤 이벤트가 state를 바꾸는가?
5. state와 props가 화면 어디에 표시되는가?

이 다섯 질문으로 대부분의 초급 React 코드를 읽을 수 있습니다.

26. 체크리스트로 보는 학습 범위

사용자가 적어준 항목을 기준으로, React 입문자가 반드시 알아야 할 범위를 다시 정리하면 아래와 같습니다.

```
JavaScript 기본 문법
배열과 객체 다루기
비동기 처리
모듈 import/export
DOM과 이벤트 개념
React 컴포넌트, JSX, props, state
입력 처리
API 연결
React Hooks
컴포넌트 분리
라우팅
```

앞에서 이미 많은 내용을 다뤘지만, 아래 섹션에서는 특히 빠지기 쉬운 JavaScript 문법, FastAPI API 호출, `axios`, `useRef`, `children`, 상태 끌어올리기, React Router까지 보강합니다.

27. JavaScript 기본 문법 보강

27.1 switch

`switch`는 값이 여러 경우 중 하나일 때 사용합니다.

```
const status = "loading";

switch (status) {
  case "loading":
    console.log("불러오는 중");
    break;
  case "success":
    console.log("성공");
    break;
  case "error":
    console.log("에러");
    break;
  default:
    console.log("알 수 없음");
}
```

React에서는 상태에 따라 다른 화면을 보여줄 때 쓸 수 있습니다.

```
function StatusMessage({ status }) {
  switch (status) {
    case "loading":
      return <p>불러오는 중...</p>;
    case "success":
      return <p>완료되었습니다.</p>;
    case "error":
      return <p>에러가 발생했습니다.</p>;
    default:
      return <p>대기 중입니다.</p>;
  }
}
```

27.2 for

for는 반복문입니다.

```
for (let i = 0; i < 3; i++) {  
  console.log(i);  
}
```

이 코드는 0, 1, 2를 출력합니다.

구조는 이렇습니다.

```
for (초기값; 반복 조건; 반복 후 실행) {  
  반복할 코드  
}
```

React에서 화면 목록을 만들 때는 보통 for보다 map을 더 많이 씁니다. 하지만 JavaScript 기본 문법으로는 for도 알아야 합니다.

27.3 for...of

for...of는 배열의 값을 하나씩 꺼낼 때 편합니다.

```
const fruits = ["사과", "바나나", "딸기"];  
  
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

이 코드는 이렇게 읽습니다.

fruits 배열에서 fruit를 하나씩 꺼낸다.

27.4 return

return은 함수가 값을 돌려주는 문법입니다.

```
function double(number) {  
  return number * 2;  
}  
  
const result = double(3);  
console.log(result); // 6
```

React 컴포넌트에서 return은 화면을 돌려줍니다.

```
function App() {  
  return <h1>화면입니다</h1>;  
}
```

return 뒤에 오는 JSX가 바로 그 컴포넌트가 보여줄 화면입니다.

27.5 템플릿 리터럴

템플릿 리터럴은 문자열 안에 JavaScript 값을 넣는 문법입니다. 백틱을 사용합니다.

```
const name = "지민";
const age = 20;

const message = `${name}님은 ${age}살입니다.`;
console.log(message);
```

React에서도 문자열을 만들 때 자주 씁니다.

```
function Profile({ name, age }) {
  return <p>`${name}님은 ${age}살입니다.`</p>;
}
```

28. 배열과 객체 다루기 보강

React에서 데이터를 다루는 힘은 대부분 배열과 객체에서 나옵니다.

28.1 forEach

forEach는 배열을 하나씩 돌면서 코드를 실행합니다.

```
const names = ["지민", "수빈", "민재"];

names.forEach((name) => {
  console.log(name);
});
```

forEach는 새 배열을 만들지 않습니다. 단순히 반복 실행만 합니다.

React에서 화면을 만들 때는 forEach보다 map을 씁니다.

```
// 화면을 만들 때는 이 방식이 더 적합합니다.
names.map((name) => <li key={name}>{name}</li>);
```

28.2 find

find는 조건에 맞는 첫 번째 항목을 찾습니다.

```
const users = [
  { id: 1, name: "지민" },
  { id: 2, name: "수빈" },
];

const user = users.find((user) => user.id === 2);

console.log(user); // { id: 2, name: "수빈" }
```

상세 페이지에서 id로 특정 데이터를 찾을 때 자주 씁니다.

28.3 reduce

reduce는 배열을 하나의 값으로 줄일 때 사용합니다.

```
const numbers = [1, 2, 3, 4];
```

```
const sum = numbers.reduce((total, number) => {
  return total + number;
}, 0);

console.log(sum); // 10
```

구조는 이렇습니다.

```
배열.reduce((누적값, 현재값) => {
  return 다음 누적값;
}, 처음 누적값)
```

장바구니 총액 계산 예시:

```
const cartItems = [
  { id: 1, name: "노트북", price: 1200000 },
  { id: 2, name: "키보드", price: 80000 },
];

const totalPrice = cartItems.reduce((total, item) => {
  return total + item.price;
}, 0);
```

28.4 객체 접근

객체 값은 두 방식으로 접근할 수 있습니다.

```
const user = {
  name: "지민",
  age: 20,
};

console.log(user.name);
console.log(user["name"]);
```

보통은 `user.name`처럼 점 표기법을 씁니다.

변수에 들어 있는 이름으로 접근해야 할 때는 대괄호를 씁니다.

```
const key = "name";

console.log(user[key]); // 지민
```

28.5 rest 문법

`rest`는 남은 값을 모을 때 사용합니다.

함수에서 사용:

```
function sum(...numbers) {
  return numbers.reduce((total, number) => total + number, 0);
}

sum(1, 2, 3); // 6
```

객체 구조 분해에서 사용:

```
const user = {
  id: 1,
  name: "지민",
  age: 20,
};

const { id, ...profile } = user;

console.log(id); // 1
console.log(profile); // { name: "지민", age: 20 }
```

`spread`는 펼치는 문법이고, `rest`는 모으는 문법이라고 생각하면 됩니다.

```
const numbers = [1, 2, 3];

const copiedNumbers = [...numbers]; // spread

function printAll(...items) { // rest
  console.log(items);
}
```

29. 비동기 처리

React에서 FastAPI 같은 서버를 호출하려면 비동기 처리를 꼭 알아야 합니다.

29.1 동기와 비동기

동기는 코드가 순서대로 끝날 때까지 기다리는 방식입니다.

```
console.log("1");
console.log("2");
console.log("3");
```

비동기는 오래 걸리는 일을 기다리는 동안 다른 코드가 먼저 실행될 수 있는 방식입니다.

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 1000);

console.log("3");
```

출력 순서는 1, 3, 2입니다.

서버 요청은 시간이 걸립니다. 그래서 `fetch`, `axios` 같은 API 호출은 비동기로 처리합니다.

29.2 Promise

`Promise`는 "나중에 결과가 생기는 값"입니다.

```
const promise = fetch("https://example.com");
```

```
console.log(promise);
```

`fetch`는 바로 결과 데이터를 주지 않고 `Promise`를 줍니다. 서버 응답이 오면 그때 결과를 사용할 수 있습니다.

29.3 `async`와 `await`

`async`와 `await`를 쓰면 비동기 코드를 순서대로 읽기 쉽게 만들 수 있습니다.

```
async function getData() {  
  const response = await fetch("https://example.com");  
  const data = await response.json();  
  
  console.log(data);  
}
```

이렇게 읽으면 됩니다.

```
fetch가 끝날 때까지 기다린다.  
응답을 JSON으로 바꿀 때까지 기다린다.  
data를 출력한다.
```

29.4 `try/catch`

`try/catch`는 에러를 처리하는 문법입니다.

```
async function getData() {  
  try {  
    const response = await fetch("https://example.com");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log("에러 발생:", error);  
  }  
}
```

API 호출에서는 거의 항상 `try/catch`를 같이 씁니다.

29.5 JSON

JSON은 서버와 클라이언트가 데이터를 주고받을 때 많이 쓰는 형식입니다.

```
{  
  "id": 1,  
  "name": "지민",  
  "age": 20  
}
```

JavaScript 객체와 비슷하지만, JSON에서는 `key`도 큰따옴표로 감쌉니다.

`fetch`로 받은 응답을 JavaScript 값으로 바꿀 때는 `response.json()`을 씁니다.

```
const response = await fetch("/api/users");  
const data = await response.json();
```

POST 요청을 보낼 때 JavaScript 객체를 JSON 문자열로 바꾸려면 `JSON.stringify`를 씁니다.

```
const body = JSON.stringify({
  name: "지민",
  age: 20,
});
```

30. HTTP 요청과 응답

HTTP는 브라우저와 서버가 대화하는 규칙입니다.

브라우저: 사용자 목록 주세요. GET /users
서버: 여기 있습니다. 200 OK + JSON 데이터

30.1 GET 요청

GET은 데이터를 가져올 때 사용합니다.

```
async function getUsers() {
  const response = await fetch("http://localhost:8000/users");
  const users = await response.json();

  console.log(users);
}
```

30.2 POST 요청

POST는 새 데이터를 만들 때 사용합니다.

```
async function createUser() {
  const response = await fetch("http://localhost:8000/users", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      name: "지민",
      age: 20,
    }),
  });

  const createdUser = await response.json();
  console.log(createdUser);
}
```

중요한 부분:

method: "POST"	POST 요청으로 보낸다.
Content-Type: application/json	JSON 형식이라고 알려준다.
body	서버에 보낼 데이터다.
JSON.stringify	JavaScript 객체를 JSON 문자열로 바꾼다.

30.3 response.ok

fetch는 서버가 404나 500을 보내도 자동으로 catch로 가지 않습니다. 그래서 response.ok를 직접 확인하는 습관이 좋습니다.

```
async function getUsers() {
  const response = await fetch("http://localhost:8000/users");

  if (!response.ok) {
    throw new Error("서버 요청 실패");
  }

  const users = await response.json();
  return users;
}
```

31. React에서 API 연결하기

React에서 API를 호출할 때는 보통 useEffect와 state를 함께 씁니다.

```
import { useEffect, useState } from "react";

export default function UserList() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchUsers() {
      try {
        setLoading(true);
        setError(null);

        const response = await fetch("http://localhost:8000/users");

        if (!response.ok) {
          throw new Error("사용자 목록을 불러오지 못했습니다.");
        }

        const data = await response.json();
        setUsers(data);
      } catch (error) {
        setError(error.message);
      } finally {
        setLoading(false);
      }
    }

    fetchUsers();
  }, []);

  if (loading) {
    return <p>불러오는 중...</p>;
  }

  if (error) {
    return <p>에러: {error}</p>;
  }
}
```



```

return (
  <ul>
    {users.map((user) => (
      <li key={user.id}>{user.name}</li>
    ))}
  </ul>
);
}

```

이 예제의 핵심:

```

users  서버에서 받아온 데이터
loading 데이터를 불러오는 중인지 여부
error  에러 메시지

```

API 화면에는 보통 세 가지 상태가 있습니다.

```

불러오는 중
성공해서 데이터 표시
실패해서 에러 표시

```

31.1 React에서 FastAPI 호출 예시

FastAPI 서버가 아래처럼 있다고 가정해 봅시다.

```

from fastapi import FastAPI

app = FastAPI()

@app.get("/todos")
def get_todos():
    return [
        {"id": 1, "text": "React 공부하기"},
        {"id": 2, "text": "FastAPI 연결하기"},
    ]

```

React에서는 이렇게 호출합니다.

```

useEffect(() => {
  async function fetchTodos() {
    const response = await fetch("http://localhost:8000/todos");
    const data = await response.json();
    setTodos(data);
  }

  fetchTodos();
}, []);

```

만약 브라우저에서 CORS 에러가 난다면 FastAPI 서버에 CORS 설정이 필요합니다.

```

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

```

```

origins = [
  "http://localhost:5173",
]

app.add_middleware(
  CORSMiddleware,
  allow_origins=origins,
  allow_credentials=True,
  allow_methods=["*"],
  allow_headers=["*"],
)

```

Vite React 개발 서버는 보통 `http://localhost:5173`에서 열립니다. 포트가 다르면 `origins`에 실제 프론트엔드 주소를 넣어야 합니다.

32. axios로 API 호출하기

`fetch`는 브라우저 기본 기능입니다. `axios`는 HTTP 요청을 편하게 해 주는 라이브러리입니다.

설치:

```
npm install axios
```

GET 요청:

```

import axios from "axios";

async function getUsers() {
  const response = await axios.get("http://localhost:8000/users");
  console.log(response.data);
}

```

POST 요청:

```

import axios from "axios";

async function createUser() {
  const response = await axios.post("http://localhost:8000/users", {
    name: "지민",
    age: 20,
  });

  console.log(response.data);
}

```

`axios`는 객체를 보내면 보통 JSON으로 자동 변환하고, 응답 JSON도 `response.data`에서 바로 사용할 수 있습니다.

React에서 `axios`를 쓰는 예시:

```

import { useEffect, useState } from "react";
import axios from "axios";

export default function UserList() {
  const [users, setUsers] = useState([]);

```

```

const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  async function fetchUsers() {
    try {
      setLoading(true);
      setError(null);

      const response = await axios.get("http://localhost:8000/users");
      setUsers(response.data);
    } catch (error) {
      setError("사용자 목록을 불러오지 못했습니다.");
    } finally {
      setLoading(false);
    }
  }

  fetchUsers();
}, []);

if (loading) {
  return <p>불러오는 중...</p>;
}

if (error) {
  return <p>{error}</p>;
}

return (
  <ul>
    {users.map((user) => (
      <li key={user.id}>{user.name}</li>
    ))}
  </ul>
);
}

```

초보자 기준으로는 먼저 `fetch`를 이해하고, 그 다음 `axios`를 쓰면 더 좋습니다. `fetch`를 알면 HTTP 요청의 기본 구조를 이해하기 쉽고, `axios`를 쓰면 실제 프로젝트 코드는 더 간결해질 수 있습니다.

33. DOM 기초와 React 이벤트

DOM은 브라우저가 HTML을 JavaScript로 다룰 수 있게 만든 구조입니다.

일반 JavaScript에서는 DOM을 직접 찾고 바꿉니다.

```

<button id="button">클릭</button>
<p id="message"></p>

```

```

const button = document.getElementById("button");
const message = document.getElementById("message");

button.addEventListener("click", () => {
  message.textContent = "버튼을 눌렀습니다.";
});

```

React에서는 DOM을 직접 조작하기보다 state를 바꿉니다.

```
import { useState } from "react";

function App() {
  const [message, setMessage] = useState("");

  return (
    <div>
      <button onClick={() => setMessage("버튼을 눌렀습니다.")}>
        클릭
      </button>
      <p>{message}</p>
    </div>
  );
}
```

이 차이가 중요합니다.

일반 JavaScript: DOM을 찾아서 직접 바꾼다.
React: state를 바꾸면 React가 화면을 다시 그린다.

34. 입력 처리 확장

React에서 입력 요소는 보통 controlled component로 만듭니다. controlled component는 입력값을 state가 관리하는 방식입니다.

34.1 textarea

```
import { useState } from "react";

function MemoForm() {
  const [memo, setMemo] = useState("");

  return (
    <div>
      <textarea
        value={memo}
        onChange={(event) => setMemo(event.target.value)}
      />
      <p>작성한 메모: {memo}</p>
    </div>
  );
}
```

34.2 select

```
import { useState } from "react";

function CategorySelect() {
  const [category, setCategory] = useState("frontend");

  return (
    <div>
      <select
```

```

    value={category}
    onChange={(event) => setCategory(event.target.value)}
  >
    <option value="frontend">프론트엔드 </option>
    <option value="backend">백엔드 </option>
    <option value="design">디자인 </option>
  </select>

  <p>선택한 분야: {category}</p>
</div>
);
}

```

34.3 여러 input 처리

여러 입력값을 객체 state 하나로 관리할 수도 있습니다.

```

import { useState } from "react";

function SignupForm() {
  const [form, setForm] = useState({
    email: "",
    password: "",
  });

  function handleChange(event) {
    const { name, value } = event.target;

    setForm({
      ...form,
      [name]: value,
    });
  }

  function handleSubmit(event) {
    event.preventDefault();
    console.log(form);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="email"
        value={form.email}
        onChange={handleChange}
        placeholder="이메일"
      />

      <input
        name="password"
        type="password"
        value={form.password}
        onChange={handleChange}
        placeholder="비밀번호"
      />

      <button type="submit">가입</button>
    </form>
  );
}

```

[name]: value는 객체의 key를 변수로 정하는 문법입니다.

```
const name = "email";

const object = {
  [name]: "hello@example.com",
};

console.log(object.email);
```

35. React Hooks 보강

35.1 useRef

useRef는 렌더링 사이에 값을 기억하지만, 값이 바뀌어도 화면을 다시 그리지 않는 저장 공간입니다.

```
import { useRef } from "react";

function ClickCounter() {
  const clickCountRef = useRef(0);

  function handleClick() {
    clickCountRef.current = clickCountRef.current + 1;
    console.log(clickCountRef.current);
  }

  return <button onClick={handleClick}>클릭</button>;
}
```

useState와 차이:

useState 값이 바뀌면 화면이 다시 렌더링된다.
useRef 값이 바뀌어도 화면이 다시 렌더링되지 않는다.

DOM 요소에 접근할 때도 씁니다.

```
import { useRef } from "react";

function FocusInput() {
  const inputRef = useRef(null);

  function focusInput() {
    inputRef.current.focus();
  }

  return (
    <div>
      <input ref={inputRef} />
      <button onClick={focusInput}>입력창 포커스</button>
    </div>
  );
}
```

초보자는 이렇게 기억하면 됩니다.

화면에 보여야 하는 값은 useState
화면에 안 보여도 기억만 하면 되는 값은 useRef
DOM 요소를 직접 잡아야 할 때도 useRef

35.2 custom hook

custom hook은 반복되는 Hook 로직을 함수로 빼는 방법입니다. 이름은 반드시 `use`로 시작합니다.

```
import { useEffect, useState } from "react";

function useUsers() {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function fetchUsers() {
      try {
        const response = await fetch("http://localhost:8000/users");
        const data = await response.json();
        setUsers(data);
      } catch (error) {
        setError("사용자 목록을 불러오지 못했습니다.");
      } finally {
        setLoading(false);
      }
    }

    fetchUsers();
  }, []);

  return { users, loading, error };
}
```

사용:

```
function UserList() {
  const { users, loading, error } = useUsers();

  if (loading) {
    return <p>불러오는 중...</p>;
  }

  if (error) {
    return <p>{error}</p>;
  }

  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

custom hook은 화면이 아니라 "상태와 로직"을 재사용할 때 만듭니다.

36. 컴포넌트 분리 보강

36.1 children

`children`은 컴포넌트 태그 사이에 들어온 내용을 받는 prop입니다.

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}

function App() {
  return (
    <Card>
      <h2>프로필</h2>
      <p>React 공부 중입니다.</p>
    </Card>
  );
}
```

이 코드는 이렇게 생각하면 됩니다.

`<Card>`와 `</Card>` 사이의 내용이 `children`으로 전달된다.

버튼, 카드, 모달, 레이아웃 컴포넌트에서 `children`을 많이 씁니다.

36.2 상위 컴포넌트와 하위 컴포넌트

상위 컴포넌트는 데이터를 가지고 있고, 하위 컴포넌트는 그 데이터를 받아서 보여주는 경우가 많습니다.

```
function TodoItem({ todo }) {
  return <li>{todo.text}</li>;
}

function TodoList() {
  const todos = [
    { id: 1, text: "React 공부하기" },
    { id: 2, text: "컴포넌트 나누기" },
  ];

  return (
    <ul>
      {todos.map((todo) => (
        <TodoItem key={todo.id} todo={todo} />
      ))}
    </ul>
  );
}
```

`TodoList`가 상위 컴포넌트이고, `TodoItem`이 하위 컴포넌트입니다.

36.3 상태 끌어올리기

두 컴포넌트가 같은 state를 함께 써야 한다면, state를 공통 부모로 올립니다. 이것을 상태 끌어올리기라고 합니다.

나쁜 구조:

`SearchInput`이 검색어 state를 가지고 있음

ProductList도 검색어가 필요함
서로 직접 값을 공유하기 어려움

좋은 구조:

App이 검색어 state를 가지고 있음
SearchInput에게 검색어와 변경 함수를 props로 전달
ProductList에게 검색어를 props로 전달

코드:

```
import { useState } from "react";

function SearchInput({ keyword, onKeywordChange }) {
  return (
    <input
      value={keyword}
      onChange={(event) => onKeywordChange(event.target.value)}
      placeholder="검색어"
    />
  );
}

function ProductList({ keyword }) {
  const products = ["노트북", "키보드", "마우스"];

  const filteredProducts = products.filter((product) =>
    product.includes(keyword)
  );

  return (
    <ul>
      {filteredProducts.map((product) => (
        <li key={product}>{product}</li>
      ))}
    </ul>
  );
}

function App() {
  const [keyword, setKeyword] = useState("");

  return (
    <main>
      <SearchInput
        keyword={keyword}
        onKeywordChange={setKeyword}
      />
      <ProductList keyword={keyword} />
    </main>
  );
}
```

핵심은 이것입니다.

여러 컴포넌트가 같은 값을 필요로 하면
그 값을 가장 가까운 공통 부모가 가진다.

37. 라우팅

라우팅은 URL에 따라 다른 페이지를 보여주는 기능입니다.

```
/      홈
/about  소개
/posts/1 1번 게시물 상세
```

React 자체에는 라우팅 기능이 없습니다. 보통 React Router를 사용합니다.

37.1 설치

현재 React Router v7 공식 문서에서는 `react-router`를 기본 import 경로로 안내합니다. 예전 강의나 v6 프로젝트에서는 `react-router-dom`을 많이 씁니다. v7의 `react-router-dom`은 v6에서 업그레이드하기 쉽게 대부분을 다시 내보내는 패키지입니다.

새 프로젝트에서는 공식 문서 기준으로 아래처럼 설치할 수 있습니다.

```
npm install react-router
```

기존 강의가 `react-router-dom`을 기준으로 한다면 아래처럼 설치해도 됩니다.

```
npm install react-router-dom
```

아래 예제는 초보자가 많이 접하는 `react-router-dom` import 기준으로 작성합니다. 만약 최신 v7 공식 예제처럼 쓰고 싶다면 import 경로만 `"react-router"`로 바꾸면 됩니다.

37.2 BrowserRouter, Routes, Route

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function HomePage() {
  return <h1>홈</h1>;
}

function AboutPage() {
  return <h1>소개</h1>;
}

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

이 코드는 이렇게 읽습니다.

BrowserRouter: 브라우저 주소를 기준으로 라우팅한다.
Routes: Route들을 감싸는 영역이다.
Route: 특정 path에서 어떤 컴포넌트를 보여줄지 정한다.

37.3 Link

일반 HTML에서는 페이지 이동에 `<a>`를 씁니다.

```
<a href="/about">소개 </a>
```

React Router에서는 보통 `<Link>`를 씁니다.

```
import { Link } from "react-router-dom";

function Navigation() {
  return (
    <nav>
      <Link to="/">홈 </Link>
      <Link to="/about">소개 </Link>
    </nav>
  );
}
```

`Link`를 쓰면 전체 페이지를 새로고침하지 않고 React 안에서 화면만 바꿉니다.

37.4 useNavigate

`useNavigate`는 코드로 페이지를 이동할 때 씁니다.

```
import { useNavigate } from "react-router-dom";

function LoginPage() {
  const navigate = useNavigate();

  function handleLogin() {
    // 로그인 성공했다고 가정
    navigate("/dashboard");
  }

  return <button onClick={handleLogin}>로그인 </button>;
}
```

버튼 클릭, 로그인 성공, 저장 완료 후 이동 같은 상황에서 사용합니다.

37.5 useParams

`useParams`는 URL의 일부 값을 꺼낼 때 씁니다.

```
import { BrowserRouter, Routes, Route, useParams } from "react-router-dom";

function PostDetailPage() {
  const params = useParams();

  return <h1>{params.postId}번 게시물 </h1>;
}

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/posts/:postId" element={<PostDetailPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

```
    </BrowserRouter>
  );
}
```

/posts/123으로 접속하면 params.postId는 "123"이 됩니다.

37.6 라우팅 파일 분리 예시

페이지가 많아지면 컴포넌트를 파일로 나눕니다.

```
src/
  App.jsx
  pages/
    HomePage.jsx
    AboutPage.jsx
    PostDetailPage.jsx
  components/
    Navigation.jsx
```

App.jsx:

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Navigation from "../components/Navigation";
import HomePage from "../pages/HomePage";
import AboutPage from "../pages/AboutPage";
import PostDetailPage from "../pages/PostDetailPage";

export default function App() {
  return (
    <BrowserRouter>
      <Navigation />

      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/about" element={<AboutPage />} />
        <Route path="/posts/:postId" element={<PostDetailPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

38. 전체 학습 순서 업데이트

사용자가 적어준 체크리스트를 실제 공부 순서로 바꾸면 이렇게 가면 좋습니다.

1. JavaScript 값, 변수, 함수, return
2. 조건문, 반복문, 배열, 객체
3. map, filter, find, reduce
4. 구조 분해 할당, spread, rest
5. import/export
6. DOM, event, input, form
7. React 컴포넌트와 JSX
8. props와 children
9. useState와 이벤트 처리
10. 조건부 렌더링과 리스트 렌더링
11. controlled component로 입력 처리

12. useEffect
13. fetch, async/await, try/catch
14. FastAPI GET/POST 연결
15. loading/error 상태 처리
16. axios
17. useRef
18. custom hook
19. 컴포넌트 파일 분리
20. 상태 끌어올리기
21. React Router

한 번에 다 외우려고 하지 않아도 됩니다. 먼저 컴포넌트, props, state, event, map만 익혀도 작은 화면은 만들 수 있습니다. 그 다음 API 연결과 라우팅으로 넘어가면 됩니다.

참고 자료

- React 공식 문서: <https://react.dev/learn>
- React API Reference: <https://react.dev/reference/react>
- React Router 공식 문서: <https://reactrouter.com>
- MDN JavaScript Guide: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- MDN Fetch API: [https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch](https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch)
- Axios 공식 문서: <https://axios-http.com/docs/intro>
- FastAPI CORS 공식 문서: <https://fastapi.tiangolo.com/tutorial/cors/>